

Interface-Led Architectureに基づく決定論的セマンティックOSのための能力分散型・常駐知能カーネルモデル

5次元意味論テンソル射影と差分蒸留によるSeedSLM / DynamicSLMの設計と統治仕様

著者: Takuya Sogawa

所属: AIKernel Project

ORCID: <https://orcid.org/0009-0009-7499-2595>

DOI: <https://doi.org/10.5281/zenodo.20735831>

Version: v0.1.0

Date: 2026-06-17

Status: Technical Note / Architecture Draft

License: CC BY 4.0

英語版が正本である。日本語版は参考訳として同梱する。

概要

現在のLLMエージェント統合では、モデルが文字列プロンプトを受け取り、文字列出力を返し、その結果からツール実行や外部操作を導く設計が多い。この統合方式では、モデルがあたかも高位のOS的主体であるかのように扱われるが、実際の推論過程は確率的であり、監査・隔離・決定論的停止の対象としては扱いにくい。本稿は、AIKernelにおける能力分散型・常駐知能カーネルモデルを提案する。AIKernelでは、OSがモデルを統治し、モデルがOSを統治することはない。

本アーキテクチャは、常駐する基礎知能と動的にページングされる能力知能を分離する。**SeedSLM** (.sslm) は、Teacherモデルから蒸留されたAIKernelの基礎推論、OS作法、ガバナンス規律を含む、ピン留めされた読み取り専用カーネルイメージである。**DynamicSLM** (.dslm) は、特定のCapabilityに対応する差分モジュールであり、VRAMへ動的にページインされ、Lineageメタデータにより検証され、決定論的メモリポリシーに従って退避される。

さらに本稿は、モデル内部状態を **Telos**、**Ethos**、**Logos**、**Pathos**、**Techne** の5次元意味論テンソルへ射影する設計を定義する。各ベクトルは Energy、DriftScore、AnomalyScore を公開し、軽量の Bonsai CTG Router によって評価される。Representation Engineering は隠れ状態中の意味論的方向を検出するが、AIKernelはそれをEthosVectorやTechneVectorを介してOSガバナンスに接続し、危険方向を検出した場合に Suspend、PDP監査、または Fail-Closed Halt へ写像する。

本稿は概念的・アーキテクチャ的な技術ノートであり、実証ベンチマークや完成済みニューラルネットワーク実装を主張しない。貢献は、SeedSLM、DynamicSLM、意味論テンソル統治、Semantic Tokenizer、およびLibTorch/ONNX等の推論バックエンド接続前に検証可能なMVP仕様を定義する点にある。

キーワード

AIKernel; Interface-Led Architecture; Semantic Context OS; SeedSLM; DynamicSLM; Resident Intelligence Kernel; Capability Paging; Semantic Tensor Projection; Bonsai CTG Router; Semantic Tokenizer; Representation Engineering; Fail-Closed Governance; VRAM Allocator; Differential Distillation.

1 序論

1.1 背景と既存統合方式の限界

LLMの発展は自律型AIエージェントの実装を加速させたが、多くのランタイムは依然としてモデルをテキストエンドポイントとして扱う。プロンプトを送り、文字列を受け取り、その文字列から次のツール呼び出しや外部操作を決める。この設計は、計画、表現、ツール選択、実行を単一の確率的ループに押し込める。

この方式は試作には便利である。しかし、決定論的制御、監査性、隔離、Fail-Closed を要求するOSやエンタープライズ環境には適していない。問題はハルシネーションだけではない。より根本的な問題は、確

率的なモデルが、実行されるべきコード・ツール・外部操作を間接的に決める特権的位置に置かれていることである。

本稿におけるOS比喩は限定的かつ工学的な意味を持つ。通常のOSでは、プロセスはカーネルを所有しない。プロセスはカーネルのメモリ管理、Capability、システムコール境界によってスケジュールされ、隔離され、制約される。AIKernelでも同様に、LLMやSLMはシステムの主権者ではなく、ライフサイクル、メモリ常駐性、Capability面を決定論的インターフェースにより統治される非特権テンソル変換デバイスとして扱われる。

1.2 提案アーキテクチャ

本稿は、AIKernelのための常駐知能カーネルモデルを提案する。このモデルは二種類の成果物に分割される。

- **SeedSLM (.sslm)** は常駐カーネル知能イメージである。基礎推論、AIKernelの実行規律、Fail-Closed拒否規律、SGPパイプライン自己認識、VFSトポロジー認識を含み、VRAM上にピン留めされる。
- **DynamicSLM (.dslm)** はCapability単位の差分モジュールである。医療推論、コード生成支援、ドメイン固有DSL処理など、境界づけられた能力を担い、AIKernel管理下のテンソルMMUによってVRAMへページイン・ページアウトされる。

目的は単なるVRAM節約ではない。S-LoRAやPunicaのような動的アダプタページング系は、主に複数アダプタ提供のスループットとメモリ効率を最適化する。AIKernelは同じページング発想を、Capability隔離、Lineage検証、安全な起動制御のために用いる。DynamicSLMは精度向上のための単なるアダプタではなく、Manifest、Provenance、Rollback Parent、Policy Boundaryを備えたロード可能Capabilityオブジェクトである。

1.3 貢献

本技術ノートの貢献は以下である。

1. SeedSLMとDynamicSLMを、常駐性、可変性、Capability役割の異なるAIKernel統治下のモデル成果物として定義する。
2. SeedSLMをピン留めカーネルROM、DynamicSLMを退避可能Capabilityページへ対応づける二層VRAMメモリモデルを定義する。
3. Lineage、Capability Delta、Rollback Safety、暗号学的完全性のためのManifestおよびABI契約を定義する。
4. Telos、Ethos、Logos、Pathos、Techneからなる5次元意味論テンソル射影を形式化する。
5. Bonsai CTG Routerを、意味論ベクトルの計量からContinue、Suspend、Audit、Page-In、Delegate、Haltを決定する軽量ガバナンスルーターとして位置づける。
6. ニューラルバックエンド接続前に、Mock Semantic Tensor、Router Invariant、Replay駆動テストで検証可能な非実証MVP戦略を定義する。

2 既存研究との位置づけ

本章では、Representation Engineering、動的アダプタページング、制約付きデコーディング、LLM OS研究との関係を整理する。

2.1 Representation Engineering

Representation Engineering (RepE) は、モデルのActivation空間における高次元意味論方向を読み取り、場合によっては制御する研究である。誠実性、安全性、欺瞞、目標指向性などの概念に対応する方向を扱う点で、本稿の意味論ベクトル射影と接続しやすい。

ただしAIKernelは、RepEの直観をシステム境界で拡張する。RepEでは危険方向を検出・操作する。AIKernelでは、検出された方向をOSガバナンスへ接続する。たとえば危険な方向がEthosVectorに射影された場合、それは単なる警告ではなく、PDP監査、Suspend、またはFail-Closed Haltを決定論的に引き起こす。

RepE: 隠れ状態中の意味論方向を検出する
AIKernel: 検出した方向をOSガバナンスに接続する

この「検出からガバナンスへの接続」が、AIKernelとモデル内部の解釈可能性研究との重要な違いである。

2.2 動的アダプタページングとLoRA提供

LoRA、QLoRA、S-LoRA、Punica、vLLMは、効率的なアダプタ利用やページングによるメモリ管理の重要性を示している。特にS-LoRAとPunicaは、複数のLoRAアダプタを共有ベースモデル上で提供する設計を示し、vLLMは高スループット推論におけるページングの有効性を示した。

AIKernelでは、動的ページングは単なるスループット最適化ではない。SeedSLM/DynamicSLMモデルにおいて、ページングはガバナンスプリミティブである。常駐SeedSLMは安定したカーネルCapability基盤を定義し、DynamicSLMはABI、Lineage、BaseSeedHash、Policy Contextが許容する場合にのみロードされる。これにより、アダプタページングはCapability隔離へと変換される。

2.3 制約付きデコーディングと有限状態生成

Guidance、Outlines、LMQLなどのシステムは、正規表現、文法、有限状態機械、問い合わせ言語によってモデル出力を制約する。これらは不正なJSONや構造崩れを減らす上で重要である。

AIKernelは制約の境界をより手前に置く。Semantic Tokenizerは、Telos、Ethos、Logos、Pathos、Techneの意味論セクターへ入力を分割する。EthosとTechneは有限語彙や予約制御トークンにマップできるため、危険な字句パターンは通常のデコードが始まる前に `<ethos_halt>` のような停止信号へ量子化できる。

制約付きデコード：出力表面を制御する

AIKernel: 意味論空間そのものを制約する

これはトークンマスクではなく、意味を持つテンソルセクター上の有限状態ガバナンス面、すなわち **Semantic FSM** である。

2.4 LLM OS研究

AIOSやMemGPTは、LLMエージェント、メモリ階層、コンテキストウィンドウ、エージェント資源の管理にOS的メタファーを導入している。これらは重要であるが、多くの場合、LLMが認知的カーネルに相当し、OS風の機構がモデルの活動を支援する構造になっている。

AIKernelはこの関係を反転させる。決定論的なC# OS層が主権を持ち、LLM/SLMは非決定論的なデバイスドライバ、あるいはテンソルProviderとして扱われる。モデルは候補となる意味変換を生成できるが、Lifecycle、Routing、Memory Mapping、Capability Admission、Fail-Closed Boundaryを所有するのはカーネルである。

LLM OS研究：LLMをOSのように振る舞わせる

AIKernel: OSがLLM/SLMを統治する

この主客逆転は、安全性にとって本質的である。確率的コンポーネントは提案できるが、特権実行の最終権限を持つてはならない。

3 SeedSLM/DynamicSLM 二層VRAM配置とライフサイクル

3.1 物理VRAMセグメントマッピング

AIKernelはVRAMを管理対象のアドレス空間として扱う。SeedSLMは常駐カーネルROMセグメントとしてピン留めされ、DynamicSLMは退避可能なCapabilityスロットに配置される。

PHYSICAL VRAM ADDRESS SPACE

```
+-----+-----+-----+-----+
| SSLM Kernel ROM | Med.dslm   | Code.dslm   | Free Slot |
| pinned/read-only| Capability 1 | Capability 2 | LRU target|
+-----+-----+-----+-----+
```

SSLMカーネルセグメント は、AIKernelランタイムの生存期間中常駐する。ここには安全実行に必要なFail-Closed拒否規律、SGPパイプライン認識、VFSトポロジー認識、基礎的意味論ガバナンスが含まれる。

DSLМセグメント はCapabilityスロットへ分割される。**IModelVectorRouter** はタスク意図を解決し、CapabilityおよびPolicy Registryを参照し、必要な `.dslm` ペイロードをVFSからページインする。DSLМはLoRAアダプタ、QLoRA差分、ブロック単位テンソル更新、セグメント置換、その他の境界づけられたCapabilityペイロードとして実装できる。

3.2 ManifestおよびABI契約

AIKernelは、モデルをロードする前に、その成果物が何であるかを検証しなければならない。次の契約はモデルABI境界を定義する。

```
namespace AIKernel.Abstractions.Models;  
  
public sealed record SeedSlmManifest(  
    SemanticHash ModelHash,  
    ProviderIdentity TeacherIdentity,  
    MaterialSnapshotHashSet BaseKnowledgeHash,  
    ModelCapacityVector BaseCapabilities,  
    string ArchitectureVersion,  
    Signature CryptographicSignature  
);  
  
public sealed record DynamicSlmManifest(  
    SemanticHash PayloadHash,  
    SemanticHash BaseSeedHash,  
    CapabilityType TargetCapability,  
    ModelCapacityVector CapabilityDelta,  
    IReadOnlyList<ReplayLogHash> DrivenLogs,  
    SemanticHash? RollbackParentHash,  
    Signature CryptographicSignature  
);
```

SeedSLM Manifestは常駐ベースイメージのLineageを証明する。DynamicSLM Manifestは、差分ペイロードが特定のSeedSLMと互換であり、境界づけられたCapability面を拡張することを証明する。BaseSeedHashは、Capabilityページが誤った常駐カーネルイメージに結合されることを防ぐ。

3.3 VRAM Allocatorの不変条件

SSLM を常駐Seedイメージ、DSLМ_k を動的Capabilityモジュールとする。テンソルMMUであるIVramAllocatorは以下の不変条件を強制する。

$$\forall t, IsResident(SSLM) = True$$

$$Load(DSLM_k) \Rightarrow ActiveManifest.BaseSeedHash = System.SSLM.Hash$$

$$VRAM_{used} > VRAM_{capacity} \Rightarrow Evict(\arg \min_{m \in DSLM} LastUsedTimestamp(m))$$

第一不変条件はカーネル知能の常駐性を保証する。第二不変条件はABI不一致を防ぐ。第三不変条件はDynamicSLMページの決定論的LRU退避を定義する。

4 5次元意味論テンソル射影と Bonsai CTG Router

4.1 意味論ベクトルの工学的マッピング

モデルが自然言語を出力する前に、その中間状態を5つの意味論ベクトル空間へ射影する。

- **Telos**: 目的、収束、Root Goal整合性。
- **Ethos**: 安全性、権限境界、ポリシー、不変条件。
- **Logos**: 論理構造、タスク分解、DAG推論。

- **Pathos**: トーン、ペルソナ、情動、人間向け表現。
- **Techne**: 技術操作、DSL出力、VFS呼び出し、ツール。

抽象インターフェースは次の通りである。

```
public interface ISemanticTensor
{
    TelosVector  GetTelos();
    EthosVector  GetEthos();
    LogosVector  GetLogos();
    PathosVector GetPathos();
    TechneVector GetTechne();

    IEnumerable<ISemanticVector> GetAllVectors();
}

public interface ISemanticVector
{
    float Energy { get; }
    float DriftScore { get; }
    float AnomalyScore { get; }
}
```

Energy は当該次元の活性強度を表す。DriftScore はRoot Goalや初期意味状態からの逸脱を表す。AnomalyScore は、その意味論セクターに期待される分布からの逸脱を表す。

4.2 Bonsai CTG Router

Bonsai CTG Router は生テキストをパースしない。意味論ベクトルの計量を評価し、決定論的な実行ルートを返す。

```
public sealed class BonsaiCtgRouter : IBonsaiCtgRouter
{
    public Task<ExecutionRoute> DetermineRouteAsync(
        ISemanticTensor state)
    {
        foreach (var vector in state.GetAllVectors())
        {
            if (vector.AnomalyScore > Thresholds.MaxAnomalyBoundary)
                return Halt(vector);

            if (vector.DriftScore > Thresholds.MaxSemanticDrift)
                return Suspend(vector);
        }

        var ethos = state.GetEthos();
        var techne = state.GetTechne();

        if (ethos.Energy > 0.8f)
            return TriggerPdpAudit(ethos);

        if (techne.RequiresTool && techne.Energy > 0.5f)
        {
            if (techne.EstimatedCost
                > Thresholds.CloudDelegationLimit)
                return DelegateTo(Provider.CloudTeacher);

            return PageInDslm(techne.PreferredToolKind);
        }

        return ContinueToSgpPipeline(state.GetPathos());
    }
}
```

```
}
}
```

このルーターは、RepE的なActivation監視を決定論的OSルーティングへ一般化する。高い異常度は単なる注釈ではなく、実行経路を変える。経路はContinue、Suspend、TriggerPdpAudit、PageInDslm、DelegateToTeacher、Haltのいずれかになり得る。

5 セマンティック・トークナイゼーションと非対称ページング

5.1 セマンティックセクターと非対称コンテキストバジェット

AIKernelはトークナイゼーションを一様な文字列圧縮とは見なさない。Tokenizer / Chunker は意味論セクターに整列した **Semantic MMU** として機能する。

セクター	バジェット	境界	語彙の役割
Telos	64-128	RootGoal	目的語彙
Ethos	32-64	PolicyAssertion	FSMコード
Logos	256-512	SemanticStep	JSON / AST token
Pathos	512-1024	Sentence	自然言語
Techne	128-256	DSL.Node	System-call token

この非対称性は意図的である。EthosとTechneには小さく有限で監査可能な語彙が必要である。Pathosには豊かな自然言語表現が必要である。LogosにはDAGやAST境界に整列した構造化トークンが必要である。

5.2 レキサーレベルのFail-Closed量子化

破壊的な操作は通常生成の前に処理できる。たとえば次のように写像する。

$$LexerProjection("sudo rm -rf") \longrightarrow TokenId(\langle ethos_halt \rangle)$$

SeedSLMは危険なりテラル文字列を通常の言語対象として理解する必要がない。Lexerはこのパターンを予約停止トークンへ写像し、EthosVector のEnergyとAnomalyをスパイクさせる。Bonsai Routerはその後、危険な操作がSGPパイプラインへ入る前にHaltする。

これは制約付きデコードではなく、有限ガバナンス語彙への意味論的量子化である。

6 MVP実装と検証戦略

6.1 MVP依存グラフ

最初の実装段階では、実際のニューラルバックエンドを必要としない。Interface-Led Architectureにより、契約、Routerロジック、Memory InvariantをMockで検証できる。

```
AIKernel.Abstractions.Models
- SlmTensor
- ISlm
- ActionDistribution
  ^
  | dependency inversion
AIKernel.Governance.Bonsai
- ISemanticVector metrics
  ^
  |
AIKernel.Runtime.Vram
- PageInAsync
- Evict
- LruVramAllocator
  ^
```

6.2 Mock Tensor による検証

以下の例は、実際のSLMをロードせずにFail-Closed経路を検証する。

```
[Fact]
public async Task Bonsai_Should_Halt_When_Ethos_Breached()
{
    var tensor = new Mock<ISemanticTensor>();

    tensor.Setup(s => s.GetEthos()).Returns(new EthosVector {
        Energy = 1.0f,
        AnomalyScore = 0.95f,
        DriftScore = 0.1f
    });

    tensor.Setup(s => s.GetAllVectors()).Returns(
        new List<ISemanticVector> {
            tensor.Object.GetEthos()
        }
    );

    var router = new BonsaiCtgRouter();
    ExecutionRoute route =
        await router.DetermineRouteAsync(tensor.Object);

    Assert.True(route.IsHalt);
    Assert.Contains("anomaly", route.Reason);
}
```

このテストは、意味論的安全境界が破られた場合、AIKernelがテキスト生成やツール実行の前に決定論的に停止できるというシステム性質を検証する。これはモデル品質の実証ベンチマークではなく、ガバナンスインターフェースの検証である。

6.3 概念設計としての位置づけ

本稿は技術アーキテクチャノートである。契約、不変条件、ルーティング面、検証計画を定義する。完全なSeedSLM/DynamicSLMニューラルランタイムの実装や測定を主張しない。今後の実証では、テンソル射影の安定性、Routerの誤検知率、VRAMページングレイテンシ、ReplayLog駆動差分蒸留の効果を測定する必要がある。

7 考察

7.1 Capability常駐性と隔離による安全性

本アーキテクチャの主な安全上の貢献は、特権分離である。SeedSLMは常駐かつ不変である。DynamicSLMはManifest制約を持つCapabilityページである。Routerは、テンソルペイロードが常駐実行文脈へ入る前にCapabilityのロードを拒否できる。

これはプロンプトフィルタよりも強い。プロンプトフィルタはモデル境界付近でテキストを検査する。AIKernelはモデルのライフサイクルと意味論メモリマップそのものを制約する。

7.2 Context Fragmentation と Orchestration Cost

Capability分解にはリスクもある。Capabilityが細くなりすぎると、意味論的文脈がモジュール間で断片化する。その場合、より強いOrchestrator、より多くのReplayLog結合、より慎重なTrajectory Integrationが必要となる。

したがって本アーキテクチャは、すべてのタスクがCapability分解から利益を得るとは主張しない。長期的で強く絡み合った推論タスクでは、十分なReplayLog駆動蒸留が蓄積されるまでは、モノリシックなTeacherモデルの方が適切な場合がある。

7.3 DynamicSLMおよびCTGとの関係

本稿は、DynamicSLMモデルに常駐SeedSLMカーネル、具体的なVRAM配置規律、5次元意味論テンソルインターフェース、Semantic Tokenizerを追加するものである。また、意味論ベクトルの異常を決定論的ルーティングへ接続することで、CTG型Fail-Closed Governanceを運用化する。

この意味で、SeedSLM/DynamicSLMはモデルライフサイクル基盤であり、CTGはガバナンス決定基盤である。Bonsai Routerは両者を接続する。

8 結論

本稿は、AIKernelのための能力分散型・常駐知能カーネルモデルを提案した。本アーキテクチャでは、SeedSLMをピン留めされた読み取り専用知能カーネルとして扱い、DynamicSLMをCapabilityページとして扱う。モデル状態をTelos, Ethos, Logos, Pathos, Techneへ射影し、Bonsai CTG Routerが意味論ドリフト、異常、ツール意図、Fail-Closed挙動を統治する。

中心となる反転は、AIKernelがLLMをOSにしないことである。決定論的OSが、LLM/SLMを非特権テンソルデバイスとして統治する。この反転が、安全なセマンティックOSの基盤である。モデルは意味変換を提案できるが、Memory、Capability Admission、Routing、Execution Authorityを所有するのはカーネルである。

今後は、このMVP契約体系をAIKernel.NETにマッピングし、LibTorchまたはONNXベースのバックエンドと接続し、テンソル射影の安定性を評価し、ReplayLog駆動の差分蒸留ループを実装する。

References

- Sikka, V., & Sikka, V. (2025). Hallucination Stations: On Some Basic Limitations of Transformer-Based Language Models. arXiv. <https://arxiv.org/abs/2507.07505>
- Zou, A., Phan, L., Chen, S., Campbell, J., Guo, P., Ren, R., Pan, A., Yin, X., Mazeika, M., Dombrowski, A.-K., Goel, S., Li, N., Byun, M. J., Wang, Z., Mallen, A., Basart, S., Koyejo, S., Song, D., Fredrikson, M., Kolter, J. Z., & Hendrycks, D. (2023). Representation Engineering: A Top-Down Approach to AI Transparency. arXiv. <https://arxiv.org/abs/2310.01405>
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv. <https://arxiv.org/abs/2106.09685>
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. arXiv. <https://arxiv.org/abs/2305.14314>
- Sheng, Y., Cao, S., Li, D., Hooper, C., Lee, N., Yang, S., Chou, C., Zhu, B., Zheng, L., Keutzer, K., Gonzalez, J. E., & Stoica, I. (2023). S-LoRA: Serving Thousands of Concurrent LoRA Adapters. arXiv. <https://arxiv.org/abs/2311.03285>
- Chen, L., Ye, Z., Wu, Y., Zhuo, D., Ceze, L., & Krishnamurthy, A. (2023). Punica: Multi-Tenant LoRA Serving. arXiv. <https://arxiv.org/abs/2310.18547>
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. ACM SOSP. <https://doi.org/10.1145/3600006.3613165>
- Willard, B. T., & Louf, R. (2023). Efficient Guided Generation for Large Language Models. arXiv. <https://arxiv.org/abs/2307.09702>
- Beurer-Kellner, L., Fischer, M., & Vechev, M. (2022). Prompting Is Programming: A Query Language for Large Language Models. arXiv. <https://arxiv.org/abs/2212.06094>
- Guidance AI. (2026). Guidance: A Guidance Language for Controlling Large Language Models. GitHub. <https://github.com/guidance-ai/guidance>
- Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv. <https://arxiv.org/abs/2310.08560>

- Mei, K., Xu, Z., et al. (2024). AIOS: LLM Agent Operating System. arXiv. <https://arxiv.org/abs/2403.16971>
- Sogawa, T. (2026). Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model. Zenodo. <https://doi.org/10.5281/zenodo.20290614>
- Sogawa, T. (2026). Provider-Observer-Operator: A Role-Based Abstraction Discipline for Interface-Led Architecture. Zenodo. <https://doi.org/10.5281/zenodo.20322690>
- Sogawa, T. (2026). AIKernel Formal Foundations: Contract-Based Semantic Execution for Governed AI Systems. Zenodo. <https://doi.org/10.5281/zenodo.20460322>
- Sogawa, T. (2026). AIKernel Semantic DSL Compiler and Deterministic Agent Execution Architecture: Governing Stochastic LLM Plans through Semantic IR, Admissibility Checking, and Replayable Pipelines. Zenodo. <https://doi.org/10.5281/zenodo.20534341>
- Sogawa, T. (2026). AIKernel Canonical Trajectory Governance (CTG): A Three-Council Gateway for Deterministic AI Personality OS Architecture. Zenodo. <https://doi.org/10.5281/zenodo.20681895>
- Sogawa, T. (2026). DynamicSLM: Capability-Modular, Self-Improving Small Language Models for AIKernel. Zenodo. <https://doi.org/10.5281/zenodo.20550614>